



Laporan Tugas Pertemuan ke-14

Identitas

- NPM : 223040024
- Nama : Diaz Alfari Rachmad
- Kelas : A
- URL Repository Github :
https://github.com/Diazalfari/pp2_223040024_A/tree/main/Latihan/sesi14/jfc-httpclient

Penjelasan Kode

Jika berkas memiliki output, gunakan tabel ini.

MainFrame.java

```
import
org.apache.http.client5.http.impl.async.CloseableHttpAsyncClient;
import org.apache.http.client5.http.impl.async.HttpAsyncClients;
import org.apache.http.core5.concurrent.FutureCallback;
import org.apache.http.core5.http.HttpHost;
import org.apache.http.core5.http.message.StatusLine;
import org.apache.http.core5.io.CloseMode;
import org.apache.http.core5.reactor.IOReactorConfig;
import org.apache.http.core5.util.Timeout;
import org.json.simple.JSONArray;
import org.json.simple.JSONObject;
import org.json.simple.parser.JSONParser;
import org.json.simple.parser.ParseException;

import javax.swing.*;
import java.awt.*;
import java.awt.event.WindowEvent;
import java.awt.event.WindowListener;
import org.apache.http.client5.http.async.methods.SimpleHttpRequest;
import org.apache.http.client5.http.async.methods.SimpleHttpResponse;
import org.apache.http.client5.http.async.methods.SimpleRequestBuilder;
import
org.apache.http.client5.http.async.methods.SimpleRequestProducer;
import
org.apache.http.client5.http.async.methods.SimpleResponseConsumer;
```



```
public class MainFrame {

    public static void main(String[] args) {
        final IOReactorConfig ioReactorConfig =
IOReactorConfig.custom()
                .setSoTimeout(Timeout.ofSeconds(5))
                .build();

        final CloseableHttpAsyncClient client =
HttpAsyncClients.custom()
                .setIOReactorConfig(ioReactorConfig)
                .build();

        client.start();

        final HttpHost target = new
HttpHost("672fbf9066e42ceaf15e9a9b.mockapi.io"); // URL target
menggunakan layanan Mock API
        final String requestUri = "/api/contacts";

        // Membuat frame utama
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Contoh HTTP Client di Swing");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(400, 200);
            frame.setLayout(new BorderLayout());

            // Label untuk status
            JLabel statusLabel = new JLabel("Tekan tombol untuk mulai
mengunduh data", JLabel.CENTER);

            // Tombol untuk memulai proses
            JButton startButton = new JButton("Mulai");

            // Progress bar
            JProgressBar progressBar = new JProgressBar(0, 100);

            // Tambahkan komponen ke frame
            frame.add(statusLabel, BorderLayout.NORTH);

            JTextArea textArea = new JTextArea();
            textArea.setEditable(false);
```



```
JScrollPane scrollPane = new JScrollPane(textArea);
frame.add(scrollPane, BorderLayout.CENTER);

JPanel panel = new JPanel();
panel.add(startButton);
panel.add(progressBar);
panel.setLayout(new FlowLayout());
frame.add(panel, BorderLayout.SOUTH);

frame.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
frame.addWindowListener(new WindowListener() {
    @Override
    public void windowOpened(WindowEvent e) {}

    @Override
    public void windowClosing(WindowEvent e) {
        client.close(CloseMode.GRACEFUL);
        System.exit(0);
    }

    @Override
    public void windowClosed(WindowEvent e) {}

    @Override
    public void windowIconified(WindowEvent e) {}

    @Override
    public void windowDeiconified(WindowEvent e) {}

    @Override
    public void windowActivated(WindowEvent e) {}

    @Override
    public void windowDeactivated(WindowEvent e) {}
});

final SimpleHttpRequest request =
SimpleRequestBuilder.get()
    .setHttpHost(target)
    .setPath(requestUri)
    .build();
```



```
// Tombol aksi
startButton.addActionListener(e -> {
    progressBar.setIndeterminate(true);
    startButton.setEnabled(false); // Nonaktifkan tombol
    saat proses berjalan
    statusLabel.setText("Proses berjalan...");
    textArea.setText("");

    client.execute(
        SimpleRequestProducer.create(request),
        SimpleResponseConsumer.create(),
        new FutureCallback<>() {
            @Override
            public void completed(final
SimpleHttpResponse response) {
                System.out.println(request + " -> " +
new StatusLine(response));

                System.out.println(response.getBodyText());

                JSONParser parser = new JSONParser();
                try {
                    JSONArray contacts = (JSONArray)
parser.parse(response.getBodyText());

                    contacts.forEach(obj -> {
                        JSONObject contact =
(JSONObject) obj;

                        String line = "Name: " +
contact.get("name") + ", Phone: " + contact.get("phone");
                        textArea.append(line + "\n");
                    });
                } catch (ParseException e) {
                    throw new RuntimeException(e);
                }

                progressBar.setIndeterminate(false);
                startButton.setEnabled(true);
                statusLabel.setText("Proses
selesai");
            }
        })
    }
```



```
@Override
public void failed(final Exception ex) {
    System.out.println(request + " ->
failed");

    progressBar.setIndeterminate(false);
    startButton.setEnabled(true);
    statusLabel.setText("Proses gagal");
}

@Override
public void cancelled() {
    System.out.println(request + " ->
cancelled");

    progressBar.setIndeterminate(false);
    startButton.setEnabled(true);
    statusLabel.setText("Proses
dibatalkan");
}

});

// Tampilkan frame
frame.setLocationRelativeTo(null);
frame.setVisible(true);
});
}
```

Penjelasan

aplikasi ini merupakan aplikasi Java Swing yang membuat HTTP client asynchronous untuk mengambil data dari API. Berikut komponen utamanya:

1. Komponen Utama:
 - Menggunakan Apache HttpComponents untuk HTTP client asynchronous
 - Interface grafis menggunakan Java Swing
 - Parser JSON untuk memproses data yang diterima
2. Antarmuka GUI terdiri dari:
 - Frame utama dengan judul "Contoh HTTP Client di Swing"
 - Label status untuk menampilkan status proses
 - Tombol "Mulai" untuk memulai pengambilan data
 - Progress bar untuk indikator proses
 - Text area untuk menampilkan hasil
3. Fitur:

- Menggunakan operasi asynchronous sehingga UI tetap responsif saat mengambil data
- Menampilkan status proses (berjalan, selesai, gagal, dibatalkan)
- Memiliki progress bar untuk indikasi visual
- Dapat menutup koneksi dengan rapi saat aplikasi ditutup

Aplikasi ini pada dasarnya adalah contoh implementasi HTTP client asynchronous dengan antarmuka grafis untuk mengambil dan menampilkan data kontak dari sebuah mock API.

Output

